Document number: P0370R2
Date: 2016-10-17
Project: C++ Extensions for Ranges, Library Working Group
Reply-to: Casey Carter <Casey@Carter.net>
Eric Niebler <Eric.Niebler@gmail.com>

Ranges TS Design Updates Omnibus

- 1 Introduction
  - 1.1 Revision history
    - 1.1.1 R1 -> R2
    - 1.1.2 R0 -> R1
- 2 Algorithms
- 3 Function -> Callable
- 4 Assignable Semantics
- 5 Customization Points
  - 5.1 Technical Specifications
- 6 Iterator/Sentinel Overhaul
  - 6.1 Technical Specifications
- 7 Implementation Experience
- 8 Acknowledgements
- References

1 Introduction

This paper presents several design changes to N4560, the working paper for the Technical Specification for C++ Extensions for Ranges (the "Ranges TS")[3]. Each change is presented herein in a separate section that motivates the change and describes the design problem that change is meant to address. The complete specification and wording of all the changes is incorporated with the wording from the working draft in P0459 "C++ Extensions for Ranges: Speculative Combined Proposals"[4].

To throw some more fuel on the fires of confusion: the first design change presented herein (customization point redesign) *is* present in N4560. However, it was not included in the text of the proposal P0021R0[5] that was voted to become the initial text of the new TS working paper in Kona. The change was "backdoored" into the WP between P0021 and the publication of N4560 in the post-Kona mailing.

1.1 Revision history

1.1.1 R1 -> R2

- Much editorial wording cleanup.
- Provide simplified wording for the specification of the `Range` and `SizedRange` concepts.
- Incorporate wording from P0459 directly instead of by reference.

1.1.2 R0 -> R1

- Remove the section proposing relaxed semantics for the comparison function objects. Further reflection on this change during the meeting, and discussions of P0307: Making Optional Greater Equal Again and P0181: Ordered By Default, make it clear that this is the wrong approach.
- Remove the empty section "Merge `Writable` and `MoveWritable`." (Which is actually proposed in P0441 "Merge `Writable` and `MoveWritable`.)
- Remove the `move_sentinel` section that discussed adapting an iterator/sentinel pair into a "move" range. LEWG felt there was not sufficient motivation to address this problem at this time. (This proposal returns in P0441 in which it is better motivated.)

2 Algorithms

There are several small issues that apply to many or all of the algorithm specifications that must be addressed by the addition of requirements in [algorithms.general]. First, an ambiguity exists for algorithms with both a two-range overload and a range + iterator overload, e.g.:

```
template <Range R1, Range R2>
void foo(R1&&, R2&&);

template <Range R, Iterator I>
void foo(R&&, I);
```

overload resolution is ambiguous when passing an array as the second parameter of `foo`, since arrays decay to pointers. Resolving this ambiguity is not complicated, but would muddy the algorithm declarations. Instead of altering the declarations, we require implementors to resolve the ambiguity in a new paragraph at the end of [algorithms.general]:

> 16 Some algorithms declare both an overload that takes a `Range` and an `Iterator`, and an overload that takes two `Range` parameters. Since an array type (3.9.2) both satisfies `Range` and decays to a pointer (4.2) which satisfies `Iterator`, such overloads are ambiguous when an array is passed as the second argument. Implementations shall provide a mechanism to resolve this ambiguity in favor of the overload that takes two ranges.

The Ranges TS adds many "range" algorithm overloads that are specified to forward to "iterator" overloads. Implementing such a range overload by directly forwarding would create inefficiencies due to introducing additional copies or moves of the arguments, e.g. `find_if` could be implemented as:

```
template<InputRange Rng, class Proj = identity,
  IndirectCallablePredicate<Projected<IteratorType<Rng>, Proj>> Pred>
safe_iterator_t<Rng>
  find_if(Rng&& rng, Pred pred, Proj proj = Proj{}) {
    return find_if(begin(rng), end(rng), std::move(pred), std::move(proj));
  }
```

which introduces moves of `pred` and `proj` that could be eliminated by perfect forwarding:

```
template<InputRange Rng, class Proj, class Pred>
  requires IndirectCallablePredicate<decay_t<Pred>, Projected<iterator_t<Rng>, decay_t<Proj>>>()
safe_iterator_t<Rng>
  find_if(Rng&& rng, Pred&& pred, Proj&& proj) {
    return find_if(begin(rng), end(rng), std::forward<Pred>(pred), std::forward<Proj>(proj));
  }

template<InputRange Rng, class Pred>
  requires IndirectCallablePredicate<decay_t<Pred>, iterator_t<Rng>>()
safe_iterator_t<Rng>
  find_if(Rng&& rng, Pred&& pred) {
    return find_if(begin(rng), end(rng), std::forward<Pred>(pred));
  }
```

except that forwarding arguments in this manner is visible to users, and so not permitted under the as-if rule: the forwarding implementation sequences the calls to begin and end *before* the actual arguments to `pred` and `proj` are copied or moved, whereas the non-forwarding implementation sequences those class *after* the argument expressions to `pred` and `proj` are copied/moved into their argument objects. To provide increased implementor freedom to perform such optimizations, and to implement the iterator/range disambiguation for arrays discussed above, we propose that the number and order of template parameters to algorithms be unspecified, and that the creation of the actual argument objects from the argument expressions be decoupled from the algorithm invocation. Such a decoupling would allow an algorithm implementation to omit or delay creation of its nominal argument objects from the actual argument expressions. These proposals can each be specified with new paragraphs in [algorithm.general]:

> 17 The number and order of template parameters for algorithm declarations is unspecified, except where explicitly stated otherwise.
>
> 18 Despite that the algorithm declarations nominally accept parameters by value, it is unspecified when and if the argument expressions are used to initialize the actual parameters except that any such initialization shall be sequenced before (1.9) the algorithm returns. [Note: The behavior of a program that modifies the values of the actual argument expressions is consequently undefined unless the algorithm return happens before (1.10) any such modifications. —end note]

These changes make both of the example implementations of `find_if` above conforming.

3 Function -> Callable

A thorough examination of the Ranges TS shows that the `Function` concept family (`Function`, `RegularFunction`, `Predicate`, `Relation`, and `StrictWeakOrder`; described in [concepts.lib.functions]) is only used in the definition of the `IndirectCallableXXX` concept family. All predicates and projections used by the algorithms are actually *callables*: object types that are evaluated via the `INVOKE` metasyntactic function. We propose to greatly simplify the specification by importing `std::invoke` from the C++ WP and replacing the `Function` concept family with a similar family of `Callable` concepts. This enables the replacement of all declarations of the form `Function<FunctionType<F>, Args...>` with `Callable<F, Args...>`, and elimination of the `as_function` machinery.

Rename section [concepts.lib.functions] to [concepts.lib.callables]; similarly rename all subsections of the form [concepts.lib.functions.Xfunction] to [concepts.lib.callables.Xcallable]. Replace the content of the section now named [concepts.lib.callables.callable]:

> 1 The `Callable` concept specifies a relationship between a callable type (20.9.1) `F` and a set of argument types `Args...` which can be evaluated by the library function `invoke` (20.9.3).

```
template <class F, class...Args>
concept bool Callable() {
  return CopyConstructible<F>() &&
    requires (F f, Args&&...args) {
      invoke(f, std::forward<Args>(args)...); // not required to be equality preserving
    };
}
```

> 2 [Note: Since the `invoke` function call expression is not required to be equality-preserving (19.1.1), a function that generates random numbers may satisfy `Callable`. —end note]

and the section [concepts.lib.callables.regularcallable]:

```
template <class F, class...Args>
concept bool RegularCallable() {
  return Callable<F, Args...>();
}
```

> 1 The `invoke` function call expression shall be equality-preserving (19.1.1). [Note: This requirement supersedes the annotation in the definition of `Callable`. —end note]
>
> 2 [Note: A random number generator does not satisfy `RegularCallable`. —end note]
>
> 3 [Note: The distinction between `Callable` and `RegularCallable` is purely semantic. —end note]

in section [concepts.lib.callables.predicate], replace references to `RegularFunction` with `RegularCallable`. In [function.objects], add to the synopsis of header `<experimental/ranges/functional>` the declaration:

```
// 20.9.3, invoke:
template <class F, class... Args>
result_of_t<F&&(Args&&...)> invoke(F&& f, Args&&... args);
```

Insert a new subsection [func.invoke] under [function.objects]:

```
template <class F, class... Args>
result_of_t<F&&(Args&&...)> invoke(F&& f, Args&&... args);
```

> 1 Effects: Equivalent to `INVOKE(std::forward<F>(f), std::forward<Args>(args)...)` (20.9.2).

Remove the section [indirectcallables.functiontype]. In [indirectcallables.indirectfunc], replace the concept definitions with:

```
template <class F, class... Is>
concept bool IndirectCallable() {
  return (Readable<Is>() && ...) &&
    Callable<F, value_type_t<Is>...>();
}

template <class F, class... Is>
concept bool IndirectRegularCallable() {
  return (Readable<Is>() && ...) &&
```

```
    RegularCallable<F, value_type_t<Is>...>();
}

template <class F, class... Is>
concept bool IndirectCallablePredicate() {
  return (Readable<Is>() && ...) &&
    Predicate<F, value_type_t<Is>...>();
}

template <class F, class I1, class I2 = I1>
concept bool IndirectCallableRelation() {
  return Readable<I1>() && Readable<I2>() &&
    Relation<F, value_type_t<I1>, value_type_t<I2>>();
}

template <class F, class I1, class I2 = I1>
concept bool IndirectCallableStrictWeakOrder() {
  return Readable<I1>() && Readable<I2>() &&
    StrictWeakOrder<F, value_type_t<I1>, value_type_t<I2>>();
}

template <class> struct indirect_result_of { };
template <class F, class... Is>
requires IndirectCallable<remove_reference_t<F>, Is...>()
struct indirect_result_of<F(Is...)> :
  result_of<F(value_type_t<Is>...)> { };
template <class F>
using indirect_result_of_t = typename indirect_result_of<F>::type;
```

Replace the definition of `projected` in [projected] with:

```
template <Readable I, IndirectRegularCallable<I> Proj>
  requires RegularCallable<Proj, reference_t<I>>()
struct projected {
  using value_type = decay_t<indirect_result_of_t<Proj&(I)>>;
  result_of_t<Proj&(reference_t<I>)> operator*() const;
};
```

In [algorithm] replace references to `Function` with `Callable`. Strike paragraph [algorithm.general]/10 that describes how the algorithms use the removed `as_function` to implement predicate and projection callables. Replace all references to `INVOKE` in the algorithm descriptions with `invoke`. Replace the descriptive text in [alg.generate] with:

> Effects: Assigns the value of `invoke(gen)` through successive iterators in the range `[first,last)`, where `last` is `first + max(n, 0)` for `generate_n`.
>
> Returns: `last`.
>
> Complexity: Exactly `last - first` evaluations of `invoke(gen)` and assignments.

4 Assignable Semantics

There seems to be some confusion in the Ranges TS about the relationship between the `Movable` and `Swappable` concepts. For example, the `Permutable` concept is required by algorithms that swap range elements, and it requires `IndirectlyMovable` instead of `IndirectlySwappable`. The specification of `swap` itself requires `Movable` elements. Does that imply that a `Movable` type `T` must satisfy `Swappable`? Certainly we experienced C++ programmers know the requirements for `std::swap` well enough that we often conflate movability with swappability.

Unfortunately, the answer to this leading question is "no." If `a` and `b` are lvalues of some `Movable` type `T`, then certainly `std::swap(a, b)` will swap the denoted values. However, `Swappable` requires that an overload found by ADL, if any, must exchange the denoted values. There is nothing in the `Movable` concept that forbids definition of a function `swap` that accepts two lvalue references to `T` and launches the missiles. We *could* redress this issue by better distinguishing move and swap operations, fastidiously requiring `Swappable` in the proper places, and attempting to better educate C++ users. However, it is our belief that this issue is so engrained in C++ culture that it would be best to make it valid.

Consequently, we propose the addition of semantic requirements to the previously purely syntactic `Assignable` concept, which when combined with `MoveConstructible` suffice to support implementation of the "default" `swap`. Specifically, we relocate the semantic requirements on the assignment expressions from `Movable` and `Copyable` to `Assignable`, which also simplifies the definitions of those two concepts. We then add a `Swappable` requirement to `Movable`, bringing `Swappable` properly into the object concept hierarchy.

Replace the content of [concepts.lib.corelang.assignable] with:

```
template <class T, class U>
concept bool Assignable() {
  return Common<T, U>() && requires(T&& t, U&& u) {
    { std::forward<T>(t) = std::forward<U>(u) } -> Same<T&>;
  };
}
```

1 Let `t` be an lvalue of type `T`, and `R` be the type `remove_reference_t<U>`. If `U` is an lvalue reference type, let `v` be an lvalue of type `R`; otherwise, let `v` be an rvalue of type `R`. Let `uu` be a distinct object of type `R` such that `uu == v`. Then `Assignable<T, U>()` is satisfied if and only if

- `std::addressof(t = v) == std::addressof(t)`.
- After evaluating `t = v`:
- `t == uu`.
- If `v` is a non-const rvalue, its resulting state is unspecified. [Note: `v` must still meet the requirements of the library component that is using it. The operations listed in those requirements must work as specified. —end note]
- Otherwise, `v` is not modified.

The entire content of [concepts.lib.object.movable] becomes:

```
template <class T>
concept bool Movable() {
  return MoveConstructible<T>() &&
    Assignable<T&, T>() &&
    Swappable<T&>();
}
```

Since the prose requirements are now redundant. As are those in [concepts.lib.object.copyable], which also now becomes simply a concept definition:

```
template <class T>
concept bool Copyable() {
  return CopyConstructible<T>() &&
    Movable<T>() &&
    Assignable<T&, const T&>();
}
```

It is now possible to change the `Movable` requirement on `exchange` in the `<experimental/ranges/utility>` header synopsis of [utility]/2 and its definition in [utility.exchange] to `MoveConstructible`:

```
template <MoveConstructible T, class U=T>
requires Assignable<T&, U>()
T exchange(T& obj, U&& new_val);
```

which suffices to implement `exchange` along with the stronger `Assignable` semantics. (A similar change could be applied to the definition of the default `swap` implementation. We don't propose this here as we've already included the effect in the definition of the `swap` customization point earlier.)

5 Customization Points

Customization points are pain points, as the motivation of N4381[2] makes clear:

> The correct usage of customization points like `swap` is to first bring the standard `swap` into scope with a using declaration, and then to call `swap` unqualified:

```
using std::swap;
swap(a, b);
```

> One problem with this approach is that it is error-prone. It is all too easy to call (qualified) `std::swap` in a generic

context, which is potentially wrong since it will fail to find any user-defined overloads.

Another potential problem – and one that will likely become bigger with the advent of Concepts Lite – is the inability to centralize constraints-checking. Suppose that a future version of `std::begin` requires that its argument model a Range concept. Adding such a constraint would have no effect on code that uses `std::begin` idiomatically:

```
using std::begin;
begin(a);
```

If the call to `begin` dispatches to a user-defined overload, then the constraint on `std::begin` has been bypassed.

This paper aims to rectify these problems by recommending that future customization points be global function objects that do argument dependent lookup internally on the users' behalf.

The range access customization points - those defined in [iterator.range], literally titled "Range Access" - should enforce "rangeness" via constraints. As constraints are pushed futher out into the "leaves" of the design, diagnostics occur closer to the point of the actual error. This design principle drives conceptification of the standard library, and it applies to customization points just as well as algorithms. Applying constraints to the customization points even enables catching errors in code that treats an argument as a range but does not properly constrain it to be so.

The same argument applies to the "Container access" customization points defined in [iterator.container] in the Working Paper. It's peculiar that these are in a distinct section from [iterator.range], since there seems to be nothing container-specific in their definitions. They seem to actually be range access customization points as well.

The Ranges TS has another customization point problem that N4381 does not cover: an implementation of the Ranges TS needs to co-exist alongside an implementation of the standard library. There's little benefit to providing customization points with strong semantic constraints if ADL can result in calls to the customization points of the same name in namespace `std`. For example, consider the definition of the single-type `Swappable` concept:

```
namespace std { namespace experimental { namespace ranges { inline namespace v1 {
  template <class T>
  concept bool Swappable() {
    return requires(T&& t, T&& u) {
      (void)swap(std::forward<T>(t), std::forward<T>(u));
    };
  }
}}}}
```

unqualified name lookup for the name `swap` could find the unconstrained `swap` in namespace `std` either directly - it's only a couple of hops up the namespace hierarchy - or via ADL if `std` is an associated namespace of `T` or `U`. If `std::swap` is unconstrained, the concept is "satisfied" for all types, and effectively useless. The Ranges TS deals with this problem by requiring changes to `std::swap`, a practice which has historically been forbidden for TSs. Applying similar constraints to all of the customization points defined in the TS by modifying the definitions in namespace `std` is an unsatisfactory solution, if not an altogether untenable.

We propose a combination of the approach used in N4381 with a "poison pill" technique to correct the lookup problem. Namely, we specify that unqualified lookup intended to find user-defined overloads via ADL must be performed in a context that includes a deleted overload matching the signature of the implementation in namespace `std`. E.g., for the customization point `begin`, the unqualified lookup for `begin(E)` (for some arbitrary expression `E`) is performed in a context that includes the declaration `void begin(const auto&) = delete;`. This "poison pill" has two distinct effects on overload resolution. First, the poison pill hides the declaration in namespace `std` from normal unqualified lookup, simply by having the same name. Second, for actual argument expressions for which the overload in namespace `std` is viable and found by ADL, the poisin pill will also be viable causing overload resolution to fail due to ambiguity. The net effect is to preclude the overload in namespace `std` from being chosen by overload resolution, or indeed any overload found by ADL that is not more specialized or more constrained than the poison pill.

All of this complicated customization point machinery is necessary to facilitate strong semantics through the addition of constraints. Let `E` be an arbitrary expression that denotes a range. The type of `begin(E)` must satisfy `Iterator`, so the customization point `begin` should constrain its return type to satisfy `Iterator`. Similarly, `end` should constrain its return type to satisfy `Sentinel<decltype(end(E)), decltype(begin(E))>()` since the iterator and sentinel types of a range must satisfy `Sentinel`. The constraints on `begin` and `end` should apply equally to the `const` and/or `reverse` variants thereof: `cbegin`, `cend`, `rbegin`, `rend`, `crbegin`, and `crend`. The size of a `SizedRange` always has a type the satisfies `Integral`, so the customization point `size` should should constrain its return type to satisfy `Integral`. `empty` should constrain its return type to be exactly `bool`. (Requiring the return type of `empty` to satisfy `Boolean` would also be a reasonable choice, but there seems to be no motivating reason for that relaxation at this time.)

5.1 Technical Specifications

Add a new subsection to the end of [type.descriptions] to introduce *customization point object* as a term of art:

> 1 A *customization point object* is a function object (20.9) with a literal class type that interacts with user-defined types while enforcing semantic requirements on that interaction.
>
> 2 The type of a customization point object shall satisfy `Semiregular` (19.4.8).
>
> 3 All instances of a specific customization point object type shall be equal.
>
> 4 The type of a customization point object `T` shall satisfy `Callable<const T, Args..>()` (19.5.2) when the types of `Args...` meet the requirements specified in that customization point object's definition. Otherwise, `T` shall not have a function call operator that participates in overload resolution.
>
> 5 Each customization point object type constrains its return type to satisfy a particular concept.
>
> 6 The library defines several named customization point objects. In every translation unit where such a name is defined, it shall refer to the same instance of the customization point object.
>
> 7 [Note: Many of the customization points objects in the library evaluate function call expressions with an unqualified name which results in a call to a user-defined function found by argument dependent name lookup (3.4.2). To preclude such an expression resulting in a call to unconstrained functions with the same name in namespace `std`, customization point objects specify that lookup for these expressions is performed in a context that includes deleted overloads matching the signatures of overloads defined in namespace `std`. When the deleted overloads are viable, user-defined overloads must be more specialized (14.5.6.2) or more constrained (Concepts TS [temp.constr.order]) to be used by a customization point object. —end note]

In [concepts.lib.corelang.swappable], replace references to `swap` in the concept definitions with references to `ranges::swap`, to make it clear that the name is used qualified here, and remove the casts to `void`:

```
template <class T>
concept bool Swappable() {
  return requires(T&& a, T&& b) {
    ranges::swap(std::forward<T>(a), std::forward<T>(b));
  };
}

template <class T, class U>
concept bool Swappable() {
  return Swappable<T>() &&
    Swappable<U>() &&
    Common<T, U>() &&
    requires(T&& t, U&& u) {
      ranges::swap(std::forward<T>(t), std::forward<U>(u));
      ranges::swap(std::forward<U>(u), std::forward<T>(t));
    };
}
```

Strike the entire note in paragraph 1 that explains the purpose of the casts to `void`.

Change paragraph 3 to read:

> An object `t` is *swappable with* an object `u` if and only if `Swappable<T, U>()` is satisfied. `Swappable<T, U>()` is satisfied if and only if given distinct objects `tt` equal to `t` and `uu` equal to `u`, after evaluating either `ranges::swap(t, u)` or `ranges::swap(u, t)`, `tt` is equal to `u` and `uu` is equal to `t`.

Strike paragraph 4.

In [utility], strike paragraph 1 (the modifications to the synposis of the standard header `utility`).

In paragraph 2, the synopsis of the header `experimental/ranges/utility`, strike `using std::swap;` and insert the text:

```
namespace {
  constexpr unspecified swap = unspecified;
}
```

Replace the entire content of [utility.swap] with:

The name `swap` denotes a customization point object (17.5.2.1.5). The effect of the expression `ranges::swap(E1, E2)` for some expressions `E1` and `E2` is equivalent to:

- `(void)swap(E1, E2)`, with overload resolution performed in a context that includes the declarations:

```
template <class T>
void swap(T&, T&) = delete;
template <class T, size_t N>
void swap(T(&)[N], T(&)[N]) = delete;
```

  and does not include a declaration of `ranges::swap`. If the function selected by overload resolution does not exchange the values denoted by `E1` and `E2`, the program is ill-formed with no diagnostic required.

- Otherwise, `(void)swap_ranges(E1, E2)` if `E1` and `E2` are lvalues of array types (3.9.2) of equal extent and `ranges::swap(*(E1), *(E2))` is a valid expression, except that `noexcept(ranges::swap(E1, E2))` is equal to `noexcept(ranges::swap(*(E1), *(E2)))`.

- Otherwise, if `E1` and `E2` are lvalues of the same type `T` which meets the syntactic requirements of `MoveConstructible<T>()` and `Assignable<T&, T>()`, exchanges the denoted values. `ranges::swap(E1, E2)` is a constant expression if the constructor selected by overload resolution for `T{std::move(E1)}` is a `constexpr` constructor and the expression `E1 = std::move(E2)` can appear in a `constexpr` function. `noexcept(ranges::swap(E1, E2))` is equal to `is_nothrow_move_constructible<T>::value && is_nothrow_move_assignable<T>::value`. If either `MoveConstructible<T>()` or `Assignable<T&, T>()` is not satisfied, the program is ill-formed with no diagnostic required. `ranges::swap(E1, E2)` has type `void`.

- Otherwise, `ranges::swap(E1, E2)` is ill-formed.

Remark: Whenever `ranges::swap(E1, E2)` is a valid expression, it exchanges the values denoted by `E1` and `E2` and has type `void`.

Note that This formulation intentionally allows swapping arrays with identical extent and differing element types, but only when swapping the element types is well-defined. Swapping arrays of `int` and `double` continues to be ill-formed, but arrays of `T` and `U` are swappable whenever `T&` and `U&` are swappable.

In [taggedtup.tagged], add to the synopsis of class template `tagged` the declaration:

```
friend void swap(tagged&, tagged&) noexcept(see below )
requires Swappable<Base&>();
```

and remove the non-member `swap` function declaration. Add paragraphs specifying the `swap` friend:

```
friend void swap(tagged& lhs, tagged& rhs) noexcept(see below )
requires Swappable<Base&>();
```

23 Remarks: The expression in the `noexcept` is equivalent to `noexcept(lhs.swap(rhs))`

24 Effects: Equivalent to: `lhs.swap(rhs)`.

25 Throws: Nothing unless the call to `lhs.swap(rhs)` throws.

Strike the section [tagged.special] that describes the non-member `swap` overload.

From [iterator.synopsis], strike the declarations of the “Range access” customization points:

```
using std::begin;
using std::end;
template <class>
  concept bool _Auto = true;
template <_Auto C> constexpr auto cbegin(const C& c) noexcept(noexcept(begin(c)))
  -> decltype(begin(c));
template <_Auto C> constexpr auto cend(const C& c) noexcept(noexcept(end(c)))
  -> decltype(end(c));
template <_Auto C> auto rbegin(C& c) -> decltype(c.rbegin());
template <_Auto C> auto rbegin(const C& c) -> decltype(c.rbegin());
template <_Auto C> auto rend(C& c) -> decltype(c.rend());
template <_Auto C> auto rend(const C& c) -> decltype(c.rend());
template <_Auto T, size_t N> reverse_iterator<T*> rbegin(T (&array)[N]);
template <_Auto T, size_t N> reverse_iterator<T*> rend(T (&array)[N]);
```

```
template <_Auto E> reverse_iterator<const E*> rbegin(initializer_list<E> il);
template <_Auto E> reverse_iterator<const E*> rend(initializer_list<E> il);
template <_Auto C> auto crbegin(const C& c) -> decltype(ranges_v1::rbegin(c));
template <_Auto C> auto crend(const C& c) -> decltype(ranges_v1::rend(c));
template <class C> auto size(const C& c) -> decltype(c.size());
template <class T, size_t N> constexpr size_t beginsize(T (&array)[N]) noexcept;
template <class E> size_t size(initializer_list<E> il) noexcept;
```

and replace with:

```
namespace {
  constexpr unspecified begin = unspecified;
  constexpr unspecified end = unspecified;
  constexpr unspecified cbegin = unspecified;
  constexpr unspecified cend = unspecified;
  constexpr unspecified rbegin = unspecified;
  constexpr unspecified rend = unspecified;
  constexpr unspecified crbegin = unspecified;
  constexpr unspecified crend = unspecified;
}
```

and under the `// 24.11, Range primitives:` comment:

```
namespace {
  constexpr unspecified size = unspecified;
  constexpr unspecified empty = unspecified;
  constexpr unspecified data = unspecified;
  constexpr unspecified cdata = unspecified;
}
```

In [ranges.range], define `iterator_t`, `sentinel_t`, and concept `Range` as:

> ```c++
> template
> using iterator_t = decltype(ranges::begin(declval()));
>
> template
> using sentinel_t = decltype(ranges::end(declval()));
>
> template
> concept bool Range() {
> return requires(T&& t) {
> ranges::end(t);
> };
> }
> ```

Strike the note immediately following (“The semantics of end (24.11.2)…”).

In [ranges.sized], replace the syntax of the `SizedRange` concept with:

```
template <class T>
concept bool SizedRange() {
  return Range<T> &&
    !disable_sized_range<remove_cv_t<remove_reference_t<T>>> &&
    requires(const remove_reference_t<T>& t) {
      { ranges::size(t) } -> ConvertibleTo<difference_type_t<iterator_t<T>>>;
    };
}
```

Replace paragraphs 3 through 5 with:

> [Note: The `disable_sized_range` predicate provides a mechanism to enable use of range types with the library that meet the syntactic requirements but do not in fact satisfy `SizedRange`. A program that instantiates a library template that requires a `Range` with such a range type `R` is ill-formed with no diagnostic required unless `disable_sized_range<remove_cv_t<remove_reference_t<R>>>` evaluates to `true` (17.5.1.3). —end note]

Strike all content from [iterator.range]. Add a new subsection [iterator.range.begin] with title `begin`:

> 1 The name `begin` denotes a customization point object (17.5.2.1.5). The effect of the expression `ranges::begin(E)` for

some expression `E` is equivalent to:

- `ranges::begin((const T&)(E))` if `E` is an rvalue of type `T`. This usage is deprecated. [Note: This deprecated usage exists so that `ranges::begin(E)` behaves similarly to `std::begin(E)` as defined in ISO/IEC 14882 when `E` is an rvalue. —end note]
- Otherwise, `(E) + 0` if `E` has array type (3.9.2).
- Otherwise, `DECAY_COPY((E).begin())` if its type `I` meets the syntactic requirements of `Iterator<I>()`. If `Iterator` is not satisfied, the program is ill-formed with no diagnostic required.
- Otherwise, `DECAY_COPY(begin(E))` if its type `I` meets the syntactic requirements of `Iterator<I>()` with overload resolution performed in a context that includes the declaration `void begin(auto&) = delete;` and does not include a declaration of `ranges::begin`. If `Iterator` is not satisfied, the program is ill-formed with no diagnostic required.
- Otherwise, `ranges::begin(E)` is ill-formed.

2 Remark: Whenever `ranges::begin(E)` is a valid expression, the type of `ranges::begin(E)` satisfies `Iterator`.

and a new subsection [iterator.range.end] with title `end`:

1 The name `end` denotes a customization point object (17.5.2.1.5). The effect of the expression `ranges::end(E)` for some expression `E` is equivalent to:

- `ranges::end((const T&)(E))` if `E` is an rvalue of type `T`. This usage is deprecated. [Note: This deprecated usage exists so that `ranges::end(E)` behaves similarly to `std::end(E)` as defined in ISO/IEC 14882 when `E` is an rvalue. —end note]
- Otherwise, `(E) + extent<T>::value` if `E` has array type (3.9.2) `T`.
- Otherwise, `DECAY_COPY((E).end())` if its type `S` meets the syntactic requirements of `Sentinel<S, decltype(ranges::begin(E)>()`. If `Sentinel` is not satisfied, the program is ill-formed with no diagnostic required.
- Otherwise, `DECAY_COPY(end(E))` if its type `S` meets the syntactic requirements of `Sentinel<S, decltype(ranges::begin(E))>()` with overload resolution performed in a context that includes the declaration `void end(auto&) = delete;` and does not include a declaration of `ranges::end`. If `Sentinel` is not satisfied, the program is ill-formed with no diagnostic required.
- Otherwise, `ranges::end(E)` is ill-formed.

2 Remark: Whenever `ranges::end(E)` is a valid expression, the types of `ranges::end(E)` and `ranges::begin(E)` satisfy `Sentinel`.

and a new subsection [iterator.range.cbegin] with title `cbegin`:

1 The name `cbegin` denotes a customization point object (17.5.2.1.5). The effect of the expression `ranges::cbegin(E)` for some expression `E` of type `T` is equivalent to `ranges::begin((const T&)(E))`.

2 Use of `ranges::cbegin(E)` with rvalue `E` is deprecated. [Note: This deprecated usage exists so that `ranges::cbegin(E)` behaves similarly to `std::cbegin(E)` as defined in ISO/IEC 14882 when `E` is an rvalue. —end note]

3 [Note: Whenever `ranges::cbegin(E)` is a valid expression, the type of `ranges::cbegin(E)` satisfies `Iterator`. —end note]

and a new subsection [iterator.range.cend] with title `cend`:

1 The name `cend` denotes a customization point object (17.5.2.1.5). The effect of the expression `ranges::cend(E)` for some expression `E` of type `T` is equivalent to `ranges::end((const T&)(E))`.

2 Use of `ranges::cend(E)` with rvalue `E` is deprecated. [Note: This deprecated usage exists so that `ranges::cend(E)` behaves similarly to `std::cend(E)` as defined in ISO/IEC 14882 when `E` is an rvalue. —end note]

3 [Note: Whenever `ranges::cend(E)` is a valid expression, the types of `ranges::cend(E)` and `ranges::cbegin(E)` satisfy `Sentinel`. —end note]

and a new subsection [iterator.range.rbegin] with title `rbegin`:

1 The name `rbegin` denotes a customization point object (17.5.2.1.5). The effect of the expression `ranges::rbegin(E)` for some expression `E` is equivalent to:

- `ranges::rbegin((const T&)(E))` if `E` is an rvalue of type `T`. This usage is deprecated. [Note: This deprecated usage exists so that `ranges::rbegin(E)` behaves similarly to `std::rbegin(E)` as defined in ISO/IEC 14882 when `E` is an rvalue. —end note]
- Otherwise, `make_reverse_iterator((E) + extent<T>::value)` if `E` has array type (3.9.2) `T`.
- Otherwise, `DECAY_COPY((E).rbegin())` if its type `I` meets the syntactic requirements of `Iterator<I>()`. If `Iterator` is not satisfied, the program is ill-formed with no diagnostic required.
- Otherwise, `make_reverse_iterator(ranges::end(E))` if both `ranges::begin(E)` and `ranges::end(E)` have the same type `I` which meets the syntactic requirements of `BidirectionalIterator<I>()` (24.2.16). If `BidirectionalIterator` is not satisfied, the program is ill-formed with no diagnostic required.
- Otherwise, `ranges::rbegin(E)` is ill-formed.

2 Remark: Whenever `ranges::rbegin(E)` is a valid expression, the type of `ranges::rbegin(E)` satisfies `Iterator`.

and a new subsection [iterator.range.rend] with title `rend`:

1 The name `rend` denotes a customization point object (17.5.2.1.5). The effect of the expression `ranges::rend(E)` for some expression `E` is equivalent to:

- `ranges::rend((const T&)(E))` if `E` is an rvalue of type `T`. This usage is deprecated. [Note: This deprecated usage exists so that `ranges::rend(E)` behaves similarly to `std::rend(E)` as defined in ISO/IEC 14882 when `E` is an rvalue. —end note]
- Otherwise, `make_reverse_iterator((E) + 0)` if `E` has array type (3.9.2).
- Otherwise, `DECAY_COPY((E).rend())` if its type `S` meets the syntactic requirements of `Sentinel<S, decltype(ranges::rbegin(E))>()`. If `Sentinel` is not satisfied, the program is ill-formed with no diagnostic required.
- Otherwise, `make_reverse_iterator(ranges::begin(E))` if both `ranges::begin(E)` and `ranges::end(E)` have the same type `I` which meets the syntactic requirements of `BidirectionalIterator<I>()` (24.2.16). If `BidirectionalIterator` is not satisfied, the program is ill-formed with no diagnostic required.
- Otherwise, `ranges::rend(E)` is ill-formed.

2 Remark: Whenever `ranges::rend(E)` is a valid expression, the types of `ranges::rend(E)` and `ranges::rbegin(E)` satisfy `Sentinel`.

and a new subsection [iterator.range.crbegin] with title `crbegin`:

1 The name `crbegin` denotes a customization point object (17.5.2.1.5). The effect of the expression `ranges::crbegin(E)` for some expression `E` of type `T` is equivalent to `ranges::rbegin((const T&)(E))`.

2 Use of `ranges::crbegin(E)` with rvalue `E` is deprecated. [Note: This deprecated usage exists so that `ranges::crbegin(E)` behaves similarly to `std::crbegin(E)` as defined in ISO/IEC 14882 when `E` is an rvalue. —end note]

3 [Note: Whenever `ranges::crbegin(E)` is a valid expression, the type of `ranges::crbegin(E)` satisfies `Iterator`. —end note]

and a new subsection [iterator.range.crend] with title `crend`:

1 The name `crend` denotes a customization point object (17.5.2.1.5). The effect of the expression `ranges::crend(E)` for some expression `E` of type `T` is equivalent to `ranges::rend((const T&)(E))`.

2 Use of `ranges::crend(E)` with rvalue `E` is deprecated. [Note: This deprecated usage exists so that `ranges::crend(E)` behaves similarly to `std::crend(E)` as defined in ISO/IEC 14882 when `E` is an rvalue. —end note]

3 [Note: Whenever `ranges::crend(E)` is a valid expression, the types of `ranges::crend(E)` and `ranges::crbegin(E)` satisfy `Sentinel`. —end note]

In [range.primitives], remove paragraphs 1-3 that define overloads of `size`. Add a new subsection [range.primitives.size] with title `size`:

1 The name `size` denotes a customization point object (17.5.2.1.5). The effect of the expression `ranges::size(E)` for some expression `E` with type `T` is equivalent to:

- `extent<T>::value` if `T` is an array type (3.9.2).
- Otherwise, `DECAY_COPY(((const T&)(E)).size())` if its type `I` satisfies `Integral<I>()` and `disable_sized_range<T>` (24.9.2.3) is `false`.
- Otherwise, `DECAY_COPY(size((const T&)(E)))` if its type `I` satisfies `Integral<I>()` with overload resolution performed in a context that includes the declaration `void size(const auto&) = delete;` and does not include a declaration of `ranges::size`, and `disable_sized_range<T>` is `false`.
- Otherwise, `DECAY_COPY(ranges::cend(E) - ranges::cbegin(E))`, except that `E` is only evaluated once, if the types `I` and `S` of `ranges::cbegin(E)` and `ranges::cend(E)` meet the syntactic requirements of `SizedSentinel<S, I>()` (24.2.9), and `ForwardIterator<I>()`. If `SizedSentinel` and `ForwardIterator` are not satisfied, the program is ill-formed with no diagnostic required.
- Otherwise, `ranges::size(E)` is ill-formed.

2 [Note: Whenever `ranges::size(E)` is a valid expression, the type of `ranges::size(E)` satisfies `Integral`. —end note]

and new subsection [range.primitives.empty] with title `empty`:

1 The name `empty` denotes a customization point object (17.5.2.1.5). The effect of the expression `ranges::empty(E)` for some expression `E` is equivalent to:

- `bool((E).empty())` if it is valid.
- Otherwise, `ranges::size(E) != 0` if it is valid.
- Otherwise, `bool(ranges::begin(E) != ranges::end(E))`, except that `E` is only evaluated once, if the type of `ranges::begin(E)` satisfies `ForwardIterator`.
- Otherwise, `ranges::empty(E)` is ill-formed.

2 Remark: Whenever `ranges::empty(E)` is a valid expression, it has type `bool`.

and a new subsection [range.primitives.data] with title `data`:

1 The name `data` denotes a customization point object (17.5.2.1.5). The effect of the expression `ranges::data(E)` for some expression `E` is equivalent to:

- `ranges::data((const T&)(E))` if `E` is an rvalue of type `T`. This usage is deprecated. [Note: This deprecated usage exists so that `ranges::data(E)` behaves similarly to `std::data(E)` as defined in the C++ Working Paper when `E` is an rvalue. —end note]
- Otherwise, `DECAY_COPY((E).data())` if it has pointer to object type.
- Otherwise, `ranges::begin(E)` if it has pointer to object type.
- Otherwise, `ranges::data(E)` is ill-formed.

2 Remark: Whenever `ranges::data(E)` is a valid expression, it has pointer to object type.

and a new subsection [range.primitives.cdata] with title `cdata`:

1 The name `cdata` denotes a customization point object (17.5.2.1.5). The effect of the expression `ranges::cdata(E)` for some expression `E` of type `T` is equivalent to `ranges::data((const T&)(E))`.

2 Use of `ranges::cdata(E)` with rvalue `E` is deprecated. [Note: This deprecated usage exists so that `ranges::cdata(E)` has behavior consistent with `ranges::data(E)` when `E` is an rvalue. —end note]

3 [Note: Whenever `ranges::cdata(E)` is a valid expression, it has pointer to object type. —end note]

6 Iterator/Sentinel Overhaul

One of the differences between the iterator model of the Ranges TS and that of Standard C++ is that the difference operation, as represented in the `SizedIteratorRange` concept, has been made semi-orthogonal to iterator category. Random access iterators *must* satisfy `SizedIteratorRange`, iterators of other categories *may* satisfy `SizedIteratorRange`. `SizedRange` provides a similar facility for ranges that know how many elements they contain, even if pairs of their iterators don't know how far apart they are. The TS has a mechanism for ranges to opt out of "sized-ness", but doesn't provide a similar mechanism for iterator and sentinel type

pairs to opt out of “sized-ness.”

Why is this even a concern? The specification of some functions in the library assumes they can be implemented to take advantage of size/distance information when available. In some cases the requirement is explicit:

```
template <Range R>
DifferenceType<IteratorType<R>> distance(R&& r);
```

1 Returns: `ranges::distance(begin(r), end(r))`

```
template <SizedRange R>
DifferenceType<IteratorType<R>> distance(R&& r);
```

2 Returns: `size(r)`

and in others, implicit:

```
template <Iterator I, Sentinel<I> S>
void advance(I& i, S bound);
```

...

7 If `SizedIteratorRange<I,S>()` is satisfied, equivalent to: `advance(i, bound - i)`.

Many of the algorithms have implementations that can take advantage of size information as well. We see three design choices for the use of size information in the library:

1. The requirement is always made explicit, as with `distance` above. A function with an alternative implementation that uses size information must be presented as an explicit overload that requires `SizedRange` or `SizedIteratorRange`. Users can see immediately whether or not they may legally pass a parameter that “looks like” it is `Sized` (i.e., meets the syntactic requirements but not the semantic requirements of the pertinent `Sized` concept) to a function.
2. Requirements can be implicit, as with `advance` above. To determine whether or not a user may legally pass a parameter that “looks like” it is `Sized`, the user examine the specification of that function and possibly the specifications of other functions that it is specified to use.
3. Make a library-wide blanket requirement that all ranges/iterator-and-sentinel pairs that meet the syntax of the pertinent `Sized` concept *must* meet the semantics.

Choices 1 & 2 suffer from the same problems. They require that the entire library be explicitly partitioned at specification time into components that may or may not use size information in their implementations. They require that users be familiar with (or have *exhaustive* knowledge for choice 2) the specifications of the library components to know which are “safe” to use. There is an enormous specification load on the library, and cognitive load on the users, to support what are essentially near-pathological corner case iterators & ranges.

Choice 3 is effectively a library-wide “duck typing” rule for a very specific case: it allows a library component to treat a parameter that is known to be a bird (e.g., `Range`) as a duck (e.g., `SizedRange`) if it looks like a duck. While this rule is also implicit, it has the advantage of being applied uniformly library-wide. We propose that choice 3 be used for the Ranges TS and that a mechanism similar to that used to opt out of `SizedRange` be provided for iterator/sentinel type pairs to opt out of `SizedIteratorRange`.

In passing, we note that the name `SizedIteratorRange` is confusing in the context of the TS, where all other `XXXRange` concepts are refinements of `Range`. Since `SizedIteratorRange` is a refinement of `Sentinel`, we think the name `SizedSentinel` is more appropriate. The template parameter order should be changed for consistency with the parameter order of `Sentinel`. Also, relocating the concept definition from its lonely section [iteratorranges.sizediteratorrange] - the sole subsection of [iteratorranges] - to be immediately after the definition of `Sentinel` produces a more comprehensible specification.

A thorough audit of iterator/sentinel semantics provides some opportunities to cleanup the language in [iterator.requirements.general], and sharpen the specifications of the iterator and sentinel concepts. Unfortunately, we notice a problem along the way: an inconsistency in the `Sentinel` semantics.

`Sentinel<S, I>()` requires `EqualityComparable<S, I>()`, which in turn requires that whenever `s1 == s2 && i1 == i2 && s1 == i1` for some values `s1, s2` and `i1, i2` of types `S` and `I`, that `s1 == i2 && s2 == i1` must also hold. Cross-type `EqualityComparable` (EC) establishes a tight correspondence between the values of the two types so that `==` is transitive across types. Cross-type EC also requires the partipant types to individually satisfy single-type EC which requires `==` to mean “equals,” i.e., substitutable in

equality-preserving expressions.

Let's try to define a sentinel type for pointers to "`int` that is less than some specified bound":

```
struct S {
  int bound;

  bool operator == (const S& that) const { return bound == that.bound; }
  bool operator != (const S& that) const { return !(*this == that); }

  friend bool operator == (const S& s, const int* p) {
    return *p >= s.bound;
  }
  friend bool operator == (const int* p, const S& s) {
    return s == p;
  }
  friend bool operator != (const int* p, const S& s) {
    return !(s == p);
  }
  friend bool operator != (const S& s, const int* p) {
    return !(s == p);
  }
};

int a[] = {1,2};
```

Is `Sentinel<S, int*>()` satisfied? Clearly the syntactic requirements are met. Consider the ranges `[a+1,S{1})` and `[a+1,S{2})`. Both `a+1 == S{1}` and `a+1 == S{2}` hold, so both ranges are empty. By cross-type EC, `(a+1 == S{1}) && (a+1 == S{2})` implies that `S{1} == S{2}` which is certainly NOT true from the definition of `S::operator==`. `S` is not a proper sentinel for `int*`.

Much of the literature around sentinels suggests that "sentinels should always compare equal." If we alter the definition of `S::operator==` so that it always returns `true`, the problem above is solved. But now consider the ranges `[a+0,S{1})` and `[a+0,S{2})`. We know from the examination of `[a+1,S{1})` and `[a+1,S{2})` that `S{1} == S{2}`. Single-type EC tells us that `S{1}` and `S{2}` must be equal (substitutable in equality-preserving expressions). But then `a+0 == S{1}` (1 >= 1) implies that `a+0 == S{2}` (1 >= 2). Another contradiction.

The principle at work here is a fundamental property of `EqualityComparable` types: any state that affects the observable behavior of an object - as witnessed by equality-preserving expressions - must participate in that object's value. Otherwise, two objects differing only in that state are `==` but NOT "equal," breaking single-type `EqualityComparable`'s requirement that `==` means "equals." We must either abandon what seems to be a large class of useful stateful sentinels, or reformulate the `Sentinel` concept to not require cross-type `EqualityComparable` and the resultant transitivity of `==`.

If we can't put sentinels into a correspondence with iterators, then sentinels must not represent *positions*. What then, are they? A perusal of the algorithms makes it clear what they require of sentinels (using `i` and `s` to denote an iterator and a sentinel):

- `i == s`, `i != s`, `s == i`, and `s != i` must all be equality-preserving expressions with the same domain
- Complement: `i != s` must be the complement of `i == s`
- Symmetry: `i == s` must be equivalent to `s == i`, and `s != i` equivalent to `i != s`
- `i == s` must be well-defined when `[i,s)` denotes a range

All requirements but the last are not particular to iterators and sentinels. We propose they be combined into a new comparison concept:

```
template <class T, class U>
concept bool WeaklyEqualityComparable() {
  return requires(const T t, const U u) {
    { t == u } -> Boolean;
    { u == t } -> Boolean;
    { t != u } -> Boolean;
    { u != t } -> Boolean;
  };
}
```

1 Let `t` and `u` be objects of types `T` and `U`. `WeaklyEqualityComparable<T, U>()` is satisfied if and only if:

- `t == u`, `u == t`, `t != u`, and `u != t` have the same domain.
- `bool(u == t) == bool(t == u)`.
- `bool(t != u) == !bool(t == u)`.

- `bool(u != t) == bool(t != u)`.

`WeaklyEqualityComparable` can then be refined by `EqualityComparable<T>`, `EqualityComparable<T, U>`, and `Sentinel`.

We also note that the algorithms don't require comparison of sentinels with sentinels; we therefore propose that sentinels be `Semiregular` instead of `Regular`.

Comparing sentinels with other sentinels isn't the only operation that is not useful to generic code: the algorithms never compare input / output iterators with input / output iterators. They only compare input / output iterators with sentinels. The reason for this is fairly obvious: input / output ranges are single-pass, so an iterator + sentinel algorithm only has access to one valid iterator value at a time; the "current" value. Obviously the "current" value is always equal to itself.

The only difference between the `Weak` and non-`Weak` variants of the `Iterator`, `InputIterator`, and `OutputIterator` concepts is the requirement for equality comparison. Why have concepts that only differ by the addition of a useless requirement? Indeed the `==` operator is slightly worse than useless since its domain is so narrow: It always either returns `true` or has undefined behavior. Of course, now that we've relaxed the Sentinel relationship the design can support "weak" ranges: ranges delimited by a "weak" iterator and a sentinel. We therefore propose that `Sentinel` be relaxed to specify the relationship between a `WeakIterator` and a `Semiregular` type that denote a range.

We also propose that `ForwardIterator<I>` be specified to refine `Sentinel<I, I>` instead of `Iterator`, after which it becomes clear that the `Iterator`, `InputIterator`, and `OutputIterator` concepts have become extraneous. The algorithms and operations can all be respecified in terms of the `Weak` variants where necessary. We propose doing so, eliminating the non-`Weak` concepts altogether, and then stripping the `Weak` prefix from the names of `WeakIterator`, `WeakInputIterator`, and `WeakOutputIterator`.

6.1 Technical Specifications

In [concepts.lib.general.equality], remove the note from paragraph 1 and replace paragraph 2 with:

> Not all input values must be valid for a given expression; e.g., for integers `a` and `b`, the expression `a / b` is not well-defined when `b` is `0`. This does not preclude the expression `a / b` being equality preserving. The *domain* of an expression is the set of input values for which the expression is required to be well-defined.

Replace [concepts.lib.compare.equalitycomparable] with:

```
template <class T, class U>
concept bool WeaklyEqualityComparable() {
  return requires(const T& t, const U& u) {
    { t == u } -> Boolean;
    { u == t } -> Boolean;
    { t != u } -> Boolean;
    { u != t } -> Boolean;
  };
}
```

Let `t` and `u` be objects of types `T` and `U`. `WeaklyEqualityComparable<T, U>()` is satisfied if and only if:

- `t == u`, `u == t`, `t != u`, and `u != t` have the same domain.
- `bool(u == t) == bool(t == u)`.
- `bool(t != u) == !bool(t == u)`.
- `bool(u != t) == bool(t != u)`.

```
template <class T>
concept bool EqualityComparable() {
  return WeaklyEqualityComparable<T, T>();
}
```

1 Let `a` and `b` be objects of type `T`. `EqualityComparable<T>()` is satisfied if and only if:

- `bool(a == b)` if and only if `a` is equal to `b`.

2 [Note: The requirement that the expression `a == b` is equality preserving implies that `==` is reflexive, transitive, and symmetric. —end note]

```
template <class T, class U>
concept bool EqualityComparable() {
  return Common<T, U>() &&
    EqualityComparable<T>() &&
```

```
    EqualityComparable<U>() &&
    EqualityComparable<common_type_t<T, U>>() &&
    WeaklyEqualityComparable<T, U>();
}
```

3 Let `a` be an object of type `T`, `b` an object of type `U`, and `C` be `common_type_t<T, U>`. Then `EqualityComparable<T, U>()` is satisfied if and only if:

- `bool(a == b) == bool(C(a) == C(b))`.

In [iterator.requirements.general]/1, strike the words “for which equality is defined” (all iterators have a difference type in the Ranges TS). In paras 2 and 3 and table 4, replace “seven” with “five” and strike references to weak input and weak output iterators. Strike the word “Weak” from para 5. Replace paras 7 through 9 with:

7 Most of the library’s algorithmic templates that operate on data structures have interfaces that use ranges. A range is an iterator and a *sentinel* that designate the beginning and end of the computation. An iterator and a sentinel denoting a range are comparable. A sentinel denotes an element when it compares equal to an iterator `i` and `i` points to that element. The types of a sentinel and an iterator that denote a range satisfy `Sentinel` (24.2.8).

8 A sentinel `s` is called *reachable* from an iterator `i` if and only if there is a finite sequence of applications of the expression `++i` that makes `i == s`. If `s` is reachable from `i`, they denote a range.

9 A range `[i,s)` is empty if `i == s`; otherwise, `[i,s)` refers to the elements in the data structure starting with the element pointed to by `i` and up to but not including the element pointed to by the first iterator `j` such that `j == s`.

10 A range `[i,s)` is valid if and only if `s` is reachable from `i`. The result of the application of functions in the library to invalid ranges is undefined.

Strike paragraph 13 (“In the following sections…”).

Strike section [iterators.iterator], and rename section [iterators.weakiterator] to [iterators.iterator]. Strike the prefix `Weak` wherever it appears in the section.

Replace the content of [iterators.sentinel] with:

1 The Sentinel concept specifies the relationship between an `Iterator` type and a `Semiregular` type whose values denote a range.

```
template <class S, class I>
concept bool Sentinel() {
  return Semiregular<S>() &&
    Iterator<I>() &&
    WeaklyEqualityComparable<S, I>();
}
```

2 Let `s` and `i` be values of type `S` and `I` such that `[i,s)` denotes a range. Types `S` and `I` satisfy `Sentinel<S, I>()` if and only if:

(2.1) — `i == s` is well-defined.

(2.2) — If `bool(i != s)` then `i` is dereferenceable and `[++i,s)` denotes a range.

3 The domain of `==` can change over time. Given an iterator `i` and sentinel `s` such that `[i,s)` denotes a range and `i != s`, `[i,s)` is not required to continue to denote a range after incrementing any iterator equal to `i`. [Note: Consequently, `i == s` is no longer required to be well-defined. —end note]

Add new subsection “Concept SizedSentinel” [iterators.sizedsentinel]:

1 The `SizedSentinel` concept specifies requirements on an `Iterator` (24.2.7) and a `Sentinel` that allow the use of the `-` operator to compute the distance between them in constant time.

```
template <class S, class I>
constexpr bool disable_sized_sentinel = false;

template <class S, class I>
concept bool SizedSentinel() {
  return Sentinel<S, I>() &&
  !disable_sized_sentinel<remove_cv_t<S>, remove_cv_t<I>> &&
  requires(const I& i, const S& s) {
```

```
    { s - i } -> Same<difference_type_t<I>>;
    { i - s } -> Same<difference_type_t<I>>;
  };
}
```

3 Let `i` be an iterator of type `I`, and `s` a sentinel of type `S` such that `[i,s)` denotes a range. Let `N` be the smallest number of applications of `++i` necessary to make `bool(i == s)` be `true`. `SizedSentinel<S, I>()` is satisfied if and only if:

(3.1) — If `N` is representable by `difference_type_t<I>`, then `s - i` is well-defined and equals `N`.

(3.2) — If `-N` is representable by `difference_type_t<I>`, then `i - s` is well-defined and equals `-N`.

4 [Note: The `disable_sized_sentinel<S, I>` predicate provides a mechanism to enable use of sentinels and iterators with the library that meet the syntactic requirements but do not in fact satisfy `SizedSentinel`. A program that instantiates a library template that requires `SizedSentinel` with an iterator type `I` and sentinel type `S` that meet the syntactic requirements of `SizedSentinel<S, I>()` but do not satisfy `SizedSentinel` is ill-formed with no diagnostic required unless `disable_sized_sentinel<S, I>` evaluates to true (17.5.1.3). —end note]

5 [Note: The `SizedSentinel` concept is satisfied by pairs of `RandomAccessIterator`s and by counted iterators and their sentinels. —end note]

Replace all references to `SizedIteratorRange<I, S>` in the document with references to `SizedSentinel<S, I>`.

Remove section [iterators.input]. Rename section [iterators.weakinput] to [iterators.input], and strip the prefix `Weak` wherever it appears.

Remove section [iterators.output]. Rename section [iterators.weakoutput] to [iterators.output], and strip the prefix `Weak` wherever it appears.

In section [iterators.forward], replace para 2 with:

2 The `ForwardIterator` concept refines `InputIterator` (24.2.11), adding equality comparison and the multi-pass guarantee, described below.

```
template <class I>
concept bool ForwardIterator() {
  return InputIterator<I>() &&
    DerivedFrom<iterator_category_t<I>, forward_iterator_tag>() &&
    Incrementable<I>() &&
    Sentinel<I, I>();
}
```

Remove section [iteratorranges].

In section [iterator.synopsis], remove the definition of `weak_input_iterator_tag`, and define `input_iterator_tag` with no bases. Strip occurrences of the prefix `Weak`. Replace the declarations delimited by the comments `// XXX Common iterators`, `// XXX Default sentinels`, `// XXX Counted iterators`, `// XXX Unreachable sentinels` with:

```
// XXX Common iterators
template <Iterator I, Sentinel<I> S>
  requires !Same<I, S>()
class common_iterator;

template <Readable I, class S>
struct value_type<common_iterator<I, S>>;

template <InputIterator I, class S>
struct iterator_category<common_iterator<I, S>>;

template <ForwardIterator I, class S>
struct iterator_category<common_iterator<I, S>>;

template <class I1, class I2, Sentinel<I2> S1, Sentinel<I1> S2>
bool operator==(
  const common_iterator<I1, S1>& x, const common_iterator<I2, S2>& y);
template <class I1, class I2, Sentinel<I2> S1, Sentinel<I1> S2>
  requires EqualityComparable<I1, I2>()
bool operator==(
  const common_iterator<I1, S1>& x, const common_iterator<I2, S2>& y);
template <class I1, class I2, Sentinel<I2> S1, Sentinel<I1> S2>
bool operator!=(
```

```
  const common_iterator<I1, S1>& x, const common_iterator<I2, S2>& y);

template <class I2, SizedSentinel<I2> I1, SizedSentinel<I2> S1, SizedSentinel<I1> S2>
difference_type_t<I2> operator-(
  const common_iterator<I1, S1>& x, const common_iterator<I2, S2>& y);

// XXX Default sentinels
class default_sentinel;

// XXX Counted iterators
template <Iterator I> class counted_iterator;

template <class I1, class I2>
  requires Common<I1, I2>()
bool operator==(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
bool operator==(
  const counted_iterator<auto>& x, default_sentinel y);
bool operator==(
  default_sentinel x, const counted_iterator<auto>& yx);
template <class I1, class I2>
  requires Common<I1, I2>()
bool operator!=(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
bool operator!=(
  const counted_iterator<auto>& x, default_sentinel y);
bool operator!=(
  default_sentinel x, const counted_iterator<auto>& y);
template <class I1, class I2>
  requires Common<I1, I2>()
bool operator<(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
template <class I1, class I2>
  requires Common<I1, I2>()
bool operator<=(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
template <class I1, class I2>
  requires Common<I1, I2>()
bool operator>(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
template <class I1, class I2>
  requires Common<I1, I2>()
bool operator>=(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
template <class I1, class I2>
  requires Common<I1, I2>()
difference_type_t<I2> operator-(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
template <class I>
  difference_type_t<I> operator-(
    const counted_iterator<I>& x, default_sentinel y);
template <class I>
  difference_type_t<I> operator-(
    default_sentinel x, const counted_iterator<I>& y);
template <RandomAccessIterator I>
  counted_iterator<I>
    operator+(difference_type_t<I> n, const counted_iterator<I>& x);
template <Iterator I>
  counted_iterator<I> make_counted_iterator(I i, difference_type_t<I> n);

template <Iterator I>
  void advance(counted_iterator<I>& i, difference_type_t<I> n);

// XXX Unreachable sentinels
class unreachable;
template <Iterator I>
  constexpr bool operator==(const I&, unreachable) noexcept;
template <Iterator I>
  constexpr bool operator==(unreachable, const I&) noexcept;
template <Iterator I>
  constexpr bool operator!=(const I&, unreachable) noexcept;
template <Iterator I>
  constexpr bool operator!=(unreachable, const I&) noexcept;
```

and replace the block of declarations in namespace `std`:

```
namespace std {
```

```
  // 24.6.2, iterator traits
  template <experimental::ranges::Iterator I>
  struct iterator_traits;
  template <experimental::ranges::InputIterator I>
  struct iterator_traits;
  template <experimental::ranges::InputIterator I>
    requires Sentinel<I, I>()
  struct iterator_traits;
}
```

In [iterator.assoc]/10, strip occurrences of the `Weak` prefix.

In [iterator.stdtraits], Add the clause `requires Sentinel<I, In>()` to the declaration of the `iterator_traits` partial specialization that has no `Weak` prefix. Strip occurrences of the `Weak` prefix.

In [std.iterator.tags], strike `weak_input_iterator_tag` as in the synopsis.

Replace the content of [iterator.operations] with:

> 24.6.5 Iterator operations [iterator.operations]
>
> 1 Since only types that satisfy `RandomAccessIterator` provide the `+` operator, and types that satisfy `SizedSentinel` provide the `-` operator, the library provides four function templates `advance`, `distance`, `next`, and `prev`. These function templates use `+` and `-` for random access iterators and ranges that satisfy `SizedSentinel`, respectively (and are, therefore, constant time for them); for output, input, forward and bidirectional iterators they use `++` to provide linear time implementations.
>
> ```
> template <Iterator I>
>   void advance(I& i, difference_type_t<I> n);
> ```
>
> 2 *Requires:* `n` shall be negative only for bidirectional iterators.
>
> 3 *Effects:* For random access iterators, equivalent to `i += n`. Otherwise, increments (or decrements for negative `n`) iterator reference `i` by `n`.
>
> ```
> template <Iterator I, Sentinel<I> S>
>   void advance(I& i, S bound);
> ```
>
> 4 *Requires:* If `Assignable<I&, S>()` is not satisfied, `[i,bound)` shall denote a range.
>
> 5 *Effects:*
>
> (5.1) — If `Assignable<I&, S>()` is satisfied, equivalent to `i = std::move(bound)`.
>
> (5.2) — Otherwise, if `SizedSentinel<S, I>()` is satisfied, equivalent to `advance(i, bound - i)`.
>
> (5.3) — Otherwise, increments `i` until `i == bound`.
>
> ```
> template <Iterator I, Sentinel<I> S>
>   difference_type_t<I> advance(I& i, difference_type_t<I> n, S bound);
> ```
>
> 6 *Requires:* If `n > 0`, `[i,bound)` shall denote a range. If `n == 0`, `[i,bound)` or `[bound,i)` shall denote a range. If `n < 0`, `[bound,i)` shall denote a range and `(BidirectionalIterator<I>() && Same<I, > S>())` shall be satisfied.
>
> 7 *Effects:*
>
> (7.1) — If `SizedSentinel<S, I>()` is satisfied:
>
> (7.1.1) — If `|n| >= |bound - i|`, equivalent to `advance(i, bound)`.
>
> (7.1.2) — Otherwise, equivalent to `advance(i, n)`.
>
> (7.2) — Otherwise, increments (or decrements for negative `n`) iterator `i` either `n` times or until `i == bound`, whichever comes first.
>
> 8 *Returns:* `n - M`, where `M` is the distance from the starting position of `i` to the ending position.
>
> ```
> template <Iterator I, Sentinel<I> S>
> ```

```
  difference_type_t<I> distance(I first, S last);
```

9 *Requires:* `[first,last)` shall denote a range, or `(Same<S, I>() && SizedSentinel<S, I>())` shall be satisfied and `[last,first)` shall denote a range.

10 *Effects:* If `SizedSentinel<S, I>()` is satisfied, returns `(last - first)`; otherwise, returns the number of increments needed to get from `first` to `last`.

```
template <Iterator I>
  I next(I x, difference_type_t<I> n = 1);
```

11 *Effects:* Equivalent to `advance(x, n); return x;`

```
template <Iterator I, Sentinel<I> S>
  I next(I x, S bound);
```

12 *Effects:* Equivalent to `advance(x, bound); return x;`

```
template <Iterator I, Sentinel<I> S>
  I next(I x, difference_type_t<I> n, S bound);
```

13 *Effects:* Equivalent to `advance(x, n, bound); return x;`

```
template <BidirectionalIterator I>
  I prev(I x, difference_type_t<I> n = 1);
```

14 *Effects:* Equivalent to `advance(x, -n); return x;`

```
template <BidirectionalIterator I>
  I prev(I x, difference_type_t<I> n, I bound);
```

15 *Effects:* Equivalent to `advance(x, -n, bound); return x;`

Replace the contents of [iterators.common] with:

24.7.4 Common iterators [iterators.common]

1 Class template `common_iterator` is an iterator/sentinel adaptor that is capable of representing a nonbounded range of elements (where the types of the iterator and sentinel differ) as a bounded range (where they are the same). It does this by holding either an iterator or a sentinel, and implementing the equality comparison operators appropriately.

2 [*Note:* The `common_iterator` type is useful for interfacing with legacy code that expects the begin and end of a range to have the same type.—*end note*]

3 [*Example:*

```
template <class ForwardIterator>
void fun(ForwardIterator begin, ForwardIterator end);

list<int> s;
// populate the list s
using CI =
  common_iterator<counted_iterator<list<int>::iterator>,
                  default_sentinel>;
// call fun on a range of 10 ints
fun(CI(make_counted_iterator(s.begin(), 10)),
    CI(default_sentinel()));
```

—*end example*]

24.7.4.1 Class template `common_iterator` [common.iterator]

```
namespace std { namespace experimental { namespace ranges { inline namespace v1 {
template <Iterator I, Sentinel<I> S>
  requires !Same<I, S>()
class common_iterator {
```

```
public:
  using difference_type = difference_type_t<I>;

  common_iterator();
  common_iterator(I i);
  common_iterator(S s);
  common_iterator(const common_iterator<ConvertibleTo<I>, ConvertibleTo<S>>& u);
  common_iterator& operator=(const common_iterator<ConvertibleTo<I>, ConvertibleTo<S>>& u);

  ~common_iterator();

  see below operator*();
  see below operator*() const;
  see below operator->() const requires Readable<I>();

  common_iterator& operator++();
  common_iterator operator++(int);

private:
  bool is_sentinel; // exposition only
  I iter;           // exposition only
  S sent;           // exposition only
};

template <Readable I, class S>
struct value_type<common_iterator<I, S>> {
  using type = value_type_t<I>;
};

template <InputIterator I, class S>
struct iterator_category<common_iterator<I, S>> {
  using type = input_iterator_tag;
};

template <ForwardIterator I, class S>
struct iterator_category<common_iterator<I, S>> {
  using type = forward_iterator_tag;
};

template <class I1, class I2, Sentinel<I2> S1, Sentinel<I1> S2>
bool operator==(
  const common_iterator<I1, S1>& x, const common_iterator<I2, S2>& y);
template <class I1, class I2, Sentinel<I2> S1, Sentinel<I1> S2>
  requires EqualityComparable<I1, I2>()
bool operator==(
  const common_iterator<I1, S1>& x, const common_iterator<I2, S2>& y);
template <class I1, class I2, Sentinel<I2> S1, Sentinel<I1> S2>
bool operator!=(
  const common_iterator<I1, S1>& x, const common_iterator<I2, S2>& y);

template <class I2, SizedSentinel<I2> I1, SizedSentinel<I2> S1, SizedSentinel<I1> S2>
difference_type_t<I2> operator-(
  const common_iterator<I1, S1>& x, const common_iterator<I2, S2>& y);
}}}}
```

1 [*Note:* It is unspecified whether `common_iterator`'s members `iter` and `sent` have distinct addresses or not. —*end note*]

24.7.4.2 `common_iterator` operations [common.iter.ops]

24.7.4.2.1 `common_iterator` constructors [common.iter.op.const]

```
common_iterator();
```

1 *Effects:* Constructs a `common_iterator`, value-initializing `is_sentinel` and `iter`. Iterator operations applied to the resulting iterator have defined behavior if and only if the corresponding operations are defined on a value-initialized iterator of type `I`.

2 *Remarks:* It is unspecified whether any initialization is performed for `sent`.

```
common_iterator(I i);
```

3 *Effects:* Constructs a `common_iterator`, initializing `is_sentinel` with `false` and `iter` with `i`.

4 *Remarks:* It is unspecified whether any initialization is performed for `sent`.

```
common_iterator(S s);
```

5 *Effects:* Constructs a `common_iterator`, initializing `is_sentinel` with `true` and `sent` with `s`.

6 *Remarks:* It is unspecified whether any initialization is performed for `iter`.

```
common_iterator(const common_iterator<ConvertibleTo<I>, ConvertibleTo<S>>& u);
```

7 *Effects:* Constructs a `common_iterator`, initializing `is_sentinel` with `u.is_sentinel`.

(7.1) — If `u.is_sentinel` is `true`, `sent` is initialized with `u.sent`.

(7.2) — If `u.is_sentinel` is `false`, `iter` is initialized with `u.iter`.

8 *Remarks:*

(8.1) — If `u.is_sentinel` is `true`, it is unspecified whether any initialization is performed for `iter`.

(8.2) — If `u.is_sentinel` is `false`, it is unspecified whether any initialization is performed for `sent`.

24.7.4.2.2 `common_iterator::operator=` [common.iter.op=]

```
common_iterator& operator=(const common_iterator<ConvertibleTo<I>, ConvertibleTo<S>>& u);
```

1 *Effects:* Assigns `u.is_sentinel` to `is_sentinel`.

(1.1) — If `u.is_sentinel` is `true`, assigns `u.sent` to `sent`.

(1.2) — If `u.is_sentinel` is `false`, assigns `u.iter` to `iter`.

Remarks:

(1.3) — If `u.is_sentinel` is `true`, it is unspecified whether any operation is performed on `iter`.

(1.4) — If `u.is_sentinel` is `false`, it is unspecified whether any operation is performed on `sent`.

2 *Returns:* `*this`

```
~common_iterator();
```

3 *Effects:* Destroys any members that are currently initialized.

24.7.4.2.3 `common_iterator::operator*` [common.iter.op.star]

```
decltype(auto) operator*();
decltype(auto) operator*() const;
```

1 *Requires:* `!is_sentinel`

2 *Effects:* Equivalent to `*iter`.

```
see below operator->() const requires Readable<I>();
```

3 *Requires:* `!is_sentinel`

4 *Effects:* Given an object `i` of type `I`

(4.1) — if `I` is a pointer type or if the expression `i.operator->()` is well-formed, this function returns `iter`.

(4.2) — Otherwise, if the expression `*iter` is a glvalue, this function is equivalent to `addressof(*iter)`.

(4.3) — Otherwise, this function returns a proxy object of an unspecified type equivalent to the following:

```
class proxy { // exposition only
  value_type_t<I> keep_;
  proxy(reference_t<I>&& x)
```

```
    : keep_(std::move(x)) {}
public:
  const value_type_t<I>* operator->() const {
    return addressof(keep_);
  }
};
```

that is initialized with `*iter`.

24.7.4.2.4 `common_iterator::operator++` [common.iter.op.incr]

```
common_iterator& operator++();
```

1 *Requires:* `!is_sentinel`

2 *Effects:* `++iter`.

3 *Returns:* `*this`.

```
common_iterator operator++(int);
```

4 *Requires:* `!is_sentinel`

5 *Effects:* Equivalent to

```
common_iterator tmp = *this;
++iter;
return tmp;
```

24.7.4.2.5 `common_iterator` comparisons [common.iter.op.comp]

```
template <class I1, class I2, Sentinel<I2> S1, Sentinel<I1> S2>
bool operator==(
  const common_iterator<I1, S1>& x, const common_iterator<I2, S2>& y);
```

1 *Effects:* Equivalent to

```
  x.is_sentinel ?
    (y.is_sentinel || y.iter == x.sent) :
    (!y.is_sentinel || x.iter == y.sent)
```

```
template <class I1, class I2, Sentinel<I2> S1, Sentinel<I1> S2>
  requires EqualityComparable<I1, I2>()
bool operator==(
  const common_iterator<I1, S1>& x, const common_iterator<I2, S2>& y);
```

2 *Effects:* Equivalent to

```
  x.is_sentinel ?
    (y.is_sentinel || y.iter == x.sent) :
    (y.is_sentinel ?
      x.iter == y.sent :
      x.iter == y.iter);
```

```
template <class I1, class I2, Sentinel<I2> S1, Sentinel<I1> S2>
bool operator!=(
  const common_iterator<I1, S1>& x, const common_iterator<I2, S2>& y);
```

3 *Effects:* Equivalent to `!(x == y)`.

```
template <class I2, SizedSentinel<I2> I1, SizedSentinel<I2> S1, SizedSentinel<I1> S2>
difference_type_t<I2> operator-(
  const common_iterator<I1, S1>& x, const common_iterator<I2, S2>& y);
```

4 *Effects:* Equivalent to

```
  x.is_sentinel ?
    (y.is_sentinel ? 0 : x.sent - y.iter) :
    (y.is_sentinel ?
      x.iter - y.sent :
      x.iter - y.iter);
```

Replace the content of [default.sentinels] with:

24.7.5 Default sentinels [default.sentinels]

24.7.5.1 Class `default_sentinel` [default.sent]

```
namespace std { namespace experimental { namespace ranges { inline namespace v1 {
  class default_sentinel { };
}}}}
```

1 Class `default_sentinel` is an empty type used to denote the end of a range. It is intended to be used together with iterator types that know the bound of their range (e.g., `counted_iterator` (24.7.6.1)).

Replace the content of [iterators.counted] with:

24.7.6 Counted iterators [iterators.counted]

1 Class template `counted_iterator` is an iterator adaptor with the same behavior as the underlying iterator except that it keeps track of its distance from its starting position. It can be used together with class `default_sentinel` in calls to generic algorithms to operate on a range of `N` elements starting at a given position without needing to know the end position *a priori*.

2 [*Example:*

```
  list<string> s;
  // populate the list s with at least 10 strings
  vector<string> v(make_counted_iterator(s.begin(), 10),
  default_sentinel()); // copies 10 strings into v
```

—*end example*]

3 Two values `i1` and `i2` of (possibly differing) types `counted_iterator<I1>` and `counted_iterator<I2>` refer to elements of the same sequence if and only if `next(i1.base(), i1.count())` and `next(i2.base(), i2.count())` refer to the same (possibly past-the-end) element.

24.7.6.1 Class template `counted_iterator` [counted.iterator]

```
namespace std { namespace experimental { namespace ranges { inline namespace v1 {
template <Iterator I>
class counted_iterator {
public:
  using iterator_type = I;
  using difference_type = difference_type_t<I>;

  counted_iterator();
  counted_iterator(I x, difference_type_t<I> n);
  counted_iterator(const counted_iterator<ConvertibleTo<I>>& i);
  counted_iterator& operator=(const counted_iterator<ConvertibleTo<I>>& i);

  I base() const;
  difference_type_t<I> count() const;
  see below operator*();
  see below operator*() const;

  counted_iterator& operator++();
  counted_iterator operator++(int);
  counted_iterator& operator--()
    requires BidirectionalIterator<I>();
  counted_iterator operator--(int)
    requires BidirectionalIterator<I>();

  counted_iterator operator+ (difference_type n) const
    requires RandomAccessIterator<I>();
  counted_iterator& operator+=(difference_type n)
    requires RandomAccessIterator<I>();
```

```
  counted_iterator operator- (difference_type n) const
    requires RandomAccessIterator<I>();
  counted_iterator& operator-=(difference_type n)
    requires RandomAccessIterator<I>();
  see below operator[](difference_type n) const
    requires RandomAccessIterator<I>();
private:
  I current;                // exposition only
  difference_type_t<I> cnt; // exposition only
};

template <Readable I>
struct value_type<counted_iterator<I>> {
  using type = value_type_t<I>;
};

template <InputIterator I>
struct iterator_category<counted_iterator<I>> {
  using type = iterator_category_t<I>;
};

template <class I1, class I2>
  requires Common<I1, I2>()
bool operator==(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
bool operator==(
  const counted_iterator<auto>& x, default_sentinel);
bool operator==(
  default_sentinel, const counted_iterator<auto>& x);

template <class I1, class I2>
  requires Common<I1, I2>()
bool operator!=(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
bool operator!=(
  const counted_iterator<auto>& x, default_sentinel y);
bool operator!=(
  default_sentinel x, const counted_iterator<auto>& y);

template <class I1, class I2>
  requires Common<I1, I2>()
bool operator<(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
template <class I1, class I2>
  requires Common<I1, I2>()
bool operator<=(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
template <class I1, class I2>
  requires Common<I1, I2>()
bool operator>(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
template <class I1, class I2>
  requires Common<I1, I2>()
bool operator>=(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
template <class I1, class I2>
  requires Common<I1, I2>()
difference_type_t<I2> operator-(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
template <class I>
difference_type_t<I> operator-(
  const counted_iterator<I>& x, default_sentinel y);
template <class I>
difference_type_t<I> operator-(
  default_sentinel x, const counted_iterator<I>& y);

template <RandomAccessIterator I>
counted_iterator<I>
  operator+(difference_type_t<I> n, const counted_iterator<I>& x);
template <Iterator I>
  counted_iterator<I> make_counted_iterator(I i, difference_type_t<I> n);
template <Iterator I>
  void advance(counted_iterator<I>& i, difference_type_t<I> n);
}}}}
```

24.7.6.2 `counted_iterator` operations [counted.iter.ops]

24.7.6.2.1 `counted_iterator` constructors [counted.iter.op.const]

```
counted_iterator();
```

1 *Effects:* Constructs a `counted_iterator`, value-initializing `current` and `cnt`. Iterator operations applied to the resulting iterator have defined behavior if and only if the corresponding operations are defined on a value-initialized iterator of type `I`.

```
counted_iterator(I i, difference_type_t<I> n);
```

2 *Requires:* `n >= 0`

3 *Effects:* Constructs a `counted_iterator`, initializing current with `i` and `cnt` with `n`.

```
counted_iterator(const counted_iterator<ConvertibleTo<I>>& i);
```

4 *Effects:* Constructs a `counted_iterator`, initializing `current` with `i.current` and `cnt` with `i.cnt`.

24.7.6.2.2 `counted_iterator::operator=` [counted.iter.op=]

```
counted_iterator& operator=(const counted_iterator<ConvertibleTo<I>>& i);
```

1 *Effects:* Assigns `i.current` to `current` and `i.cnt` to `cnt`.

24.7.6.2.3 `counted_iterator` conversion [counted.iter.op.conv]

```
I base() const;
```

1 *Returns:* `current`.

24.7.6.2.4 `counted_iterator` count [counted.iter.op.cnt]

```
difference_type_t<I> count() const;
```

1 *Returns:* `cnt`.

24.7.6.2.5 `counted_iterator::operator*` [counted.iter.op.star]

```
decltype(auto) operator*();
decltype(auto) operator*() const;
```

1 *Effects:* Equivalent to `*current`.

24.7.6.2.6 `counted_iterator::operator++` [counted.iter.op.incr]

```
counted_iterator& operator++();
```

1 *Requires:* `cnt > 0`
2 *Effects:* Equivalent to

```
  ++current;
  --cnt;
```

3 *Returns:* `*this`.

```
counted_iterator operator++(int);
```

4 *Requires:* `cnt > 0`
5 *Effects:* Equivalent to

```
  counted_iterator tmp = *this;
  ++current;
  --cnt;
```

```
  return tmp;
```

24.7.6.2.7 `counted_iterator::operator--` [counted.iter.op.decr]

```
counted_iterator& operator--();
  requires BidirectionalIterator<I>()
```

1 *Effects:* Equivalent to

```
  --current;
  ++cnt;
```

2 *Returns:* `*this`.

```
counted_iterator operator--(int)
  requires BidirectionalIterator<I>();
```

3 *Effects:* Equivalent to

```
  counted_iterator tmp = *this;
  --current;
  ++cnt;
  return tmp;
```

24.7.6.2.8 `counted_iterator::operator+` [counted.iter.op.+]

```
counted_iterator operator+(difference_type n) const
  requires RandomAccessIterator<I>();
```

1 *Requires:* `n <= cnt`

2 *Effects:* Equivalent to `counted_iterator(current + n, cnt - n)`.

24.7.6.2.9 `counted_iterator::operator+=` [counted.iter.op.+=]

```
counted_iterator& operator+=(difference_type n)
  requires RandomAccessIterator<I>();
```

1 *Requires:* `n <= cnt`

2 *Effects:*

```
  current += n;
  cnt -= n;
```

3 *Returns:* `*this`.

24.7.6.2.10 `counted_iterator::operator-` [counted.iter.op.-]

```
counted_iterator operator-(difference_type n) const
  requires RandomAccessIterator<I>();
```

1 *Requires:* `-n <= cnt`

2 *Effects:* Equivalent to `counted_iterator(current - n, cnt + n)`.

24.7.6.2.11 `counted_iterator::operator-=` [counted.iter.op.-=]

```
counted_iterator& operator-=(difference_type n)
  requires RandomAccessIterator<I>();
```

1 *Requires:* `-n <= cnt`

2 *Effects:*

```
  current -= n;
  cnt += n;
```

3 *Returns:* `*this`.

24.7.6.2.12 `counted_iterator::operator[]` [counted.iter.op.index]

```
decltype(auto) operator[](difference_type n) const
  requires RandomAccessIterator<I>();
```

1 *Requires:* `n <= cnt`

2 *Effects:* Equivalent to `current[n]`.

24.7.6.2.13 `counted_iterator` comparisons [counted.iter.op.comp]

```
template <class I1, class I2>
  requires Common<I1, I2>()
bool operator==(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
```

1 *Requires:* `x` and `y` shall refer to elements of the same sequence (24.7.6).

2 *Effects:* Equivalent to `x.cnt == y.cnt`.

```
bool operator==(
  const counted_iterator<auto>& x, default_sentinel);
bool operator==(
  default_sentinel, const counted_iterator<auto>& x);
```

3 *Effects:* Equivalent to `x.cnt == 0`.

```
template <class I1, class I2>
  requires Common<I1, I2>()
bool operator!=(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
bool operator!=(
  const counted_iterator<auto>& x, default_sentinel);
bool operator!=(
  default_sentinel, const counted_iterator<auto>& x);
```

4 *Requires:* For the first overload, `x` and `y` shall refer to elements of the same sequence (24.7.6).

5 *Effects:* Equivalent to `!(x == y)`.

```
template <class I1, class I2>
  requires Common<I1, I2>()
bool operator<(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
```

6 *Requires:* `x` and `y` shall refer to elements of the same sequence (24.7.6).

7 *Effects:* Equivalent to `y.cnt < x.cnt`.

Remark: The argument order in the Effects element is reversed because `cnt` counts down, not up.

```
template <class I1, class I2>
  requires Common<I1, I2>()
bool operator<=(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
```

8 *Requires:* `x` and `y` shall refer to elements of the same sequence (24.7.6).

9 *Effects:* Equivalent to `!(y < x)`.

```
template <class I1, class I2>
  requires Common<I1, I2>()
bool operator>(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
```

10 *Requires:* `x` and `y` shall refer to elements of the same sequence (24.7.6).

11 *Effects:* Equivalent to `y < x`.

```
template <class I1, class I2>
  requires Common<I1, I2>()
bool operator>=(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
```

12 *Requires:* `x` and `y` shall refer to elements of the same sequence (24.7.6).

13 *Effects:* Equivalent to `!(x < y)`.

24.7.6.2.14 `counted_iterator` non-member functions [counted.iter.nonmember]

```
template <class I1, class I2>
  requires Common<I1, I2>()
difference_type_t<I2> operator-(
  const counted_iterator<I1>& x, const counted_iterator<I2>& y);
```

1 *Requires:* `x` and `y` shall refer to elements of the same sequence (24.7.6).

2 *Effects:* Equivalent to `y.cnt - x.cnt`.

```
template <class I>
difference_type_t<I> operator-(
  const counted_iterator<I>& x, default_sentinel y);
```

3 *Effects:* Equivalent to `-x.cnt`.

```
template <class I>
difference_type_t<I> operator-(
  default_sentinel x, const counted_iterator<I>& y);
```

4 *Effects:* Equivalent to `y.cnt`.

```
template <RandomAccessIterator I>
counted_iterator<I>
  operator+(difference_type_t<I> n, const counted_iterator<I>& x);
```

5 *Requires:* `n <= x.cnt`.

6 *Effects:* Equivalent to `x + n`.

```
template <Iterator I>
counted_iterator<I> make_counted_iterator(I i, difference_type_t<I> n);
```

7 *Requires:* `n >= 0`.

8 *Returns:* `counted_iterator<I>(i, n)`.

```
template <Iterator I>
void advance(counted_iterator<I>& i, difference_type_t<I> n);
```

9 *Requires:* `n <= i.cnt`.
10 *Effects:*

```
  i = make_counted_iterator(next(i.current, n), i.cnt - n);
```

Replace the content of [unreachable.sentinels] with:

24.7.8 Unreachable sentinel [unreachable.sentinels]

24.7.8.1 Class `unreachable` [unreachable.sentinel]

1 Class `unreachable` is a sentinel type that can be used with any `Iterator` to denote an infinite range. Comparing an iterator for equality with an object of type `unreachable` always returns `false`.

[*Example:*

```
// set p to point to a character buffer containing newlines
char* nl = find(p, unreachable(), '\n');
```

Provided a newline character really exists in the buffer, the use of `unreachable` above potentially makes the call to find more efficient since the loop test against the sentinel does not require a conditional branch.

—*end example*]

```
namespace std { namespace experimental { namespace ranges { inline namespace v1 {
class unreachable { };

template <Iterator I>
  constexpr bool operator==(const I&, unreachable) noexcept;
template <Iterator I>
  constexpr bool operator==(unreachable, const I&) noexcept;
template <Iterator I>
  constexpr bool operator!=(const I&, unreachable) noexcept;
template <Iterator I>
  constexpr bool operator!=(unreachable, const I&) noexcept;
}}}}
```

24.7.8.2 `unreachable` operations [unreachable.sentinel.ops]

24.7.8.2.1 `operator==` [unreachable.sentinel.op==]

```
template <Iterator I>
  constexpr bool operator==(const I&, unreachable) noexcept;
template <Iterator I>
  constexpr bool operator==(unreachable, const I&) noexcept;
```

1 *Returns:* `false`.

24.7.8.2.2 `operator!=` [unreachable.sentinel.op!=]

```
template <Iterator I>
  constexpr bool operator!=(const I& x, unreachable y) noexcept;
template <Iterator I>
  constexpr bool operator!=(unreachable x, const I& y) noexcept;
```

1 *Returns:* `true`.

Strip occurrences of the prefix `Weak` wherever it appears in clause [algorithms].

7 Implementation Experience

The proposed design changes are implemented in CMCSTL2, a full implementation of the Ranges TS with proxy extensions[1].

8 Acknowledgements

The authors would like to thank Andrew Sutton and Sean Parent for their participation in the discussions that produced most of the ideas herein.

References

[1] CMCSTL2: *https://github.com/CaseyCarter/cmcstl2*. Accessed: 2016-10-15.

[2] Niebler, E. 2015. Suggested Design for Customization Points.

[3] Niebler, E. and Carter, C. 2015. N4560: Working Draft: C++ Extensions for Ranges.

[4] Niebler, E. and Carter, C. 2016. P0459R0: C++ Extensions for Ranges, Speculative Combined Proposals.

[5] Niebler, E. and Carter, C. 2015. Working Draft, C++ Extensions for Ranges.